AIML ENGINEERING LAB

AWS Cloud Infrastructure Guide

Complete reference for P053 Memory Yield Predictor — all services, commands, costs, security, and operational procedures.

TABLE OF CONTENTS

P053 — AWS Cloud Infrastructure

1 Architecture Overview
2 Service Inventory
  2.1 EC2 — g4dn.xlarge (GPU Compute)
  2.2 RDS — PostgreSQL 17 (MLflow Backend)
  2.3 S3 — Artifact Storage
  2.4 ECR — Docker Image Registry
  2.5 Security Group & Networking
  2.6 CloudWatch — Billing Alarm
3 Setup Walkthrough (Steps 0–13)
  3.1 Prerequisites
  3.2 Phase 2 — EC2 + RDS Provisioning
  3.3 Phase 3 — 40-Day Simulation
4 GPU Quota — Issue & Resolution
5 Cost Tracking & Control
6 Security Practices
7 Operational Procedures
  7.1 Stop Everything (Save Cost)
  7.2 Resume When Quota Approved
  7.3 MacBook Requirement
  7.4 RDS-Managed Elastic IPs
8 Quick Reference Commands

1 Architecture Overview

> **Principle:** EVERYTHING on AWS. Zero MacBook training. Zero Colab retrains. All data on S3. All experiments in MLflow PostgreSQL RDS. Auto-stop EC2 + RDS + NAT after simulation. $200 CloudWatch billing alarm.
>
> **Account:** 718036735422 | **Region:** us-west-2 (Oregon) | **Budget:** ~$72 USD (full 40-day simulation)

Layer	Services	Flows To / Purpose
Compute	EC2 g4dn.xlarge (T4 GPU, 16 GB VRAM)	Runs Docker Compose: Airflow, MLflow server, FastAPI, Prometheus, Grafana
Metadata Store	RDS PostgreSQL 17 (db.t3.micro)	MLflow experiment runs, metrics, parameters, model registry — stopped when idle
Artifact Store	S3 (p053-mlflow-artifacts, versioned)	Model weights (.pt), training data (.npz), drift reports (.json)
Container Registry	ECR (053-memory-yield-predictor)	GPU-enabled Airflow Docker image with PyTorch + CUDA 12.1
Networking	Security Group sg-0f11ba29c1155cba3	6 ports: SSH(22), MLflow(5001), FastAPI(8000), Prometheus(9090), Grafana(3000), Airflow(8080)
Monitoring	CloudWatch Billing Alarm	$200 USD threshold — notifies before overspend
CI/CD	GitHub Actions + ECR	lint → test → Docker build → ECR push on git tag

Data flow during 40-day simulation:

Airflow DAG (on EC2) generates daily DRAM data → detects drift (PSI + KL + KS) → if drift confirmed: trains HybridTransformerCNN on T4 GPU via `train.py` subprocess → logs metrics to MLflow (RDS backend) → saves model to S3 → canary comparison → promote/rollback in Model Registry → FastAPI serves new model → Prometheus scrapes → Grafana displays → ec2_auto_stop.py shuts everything down after Phase 3.

2 Service Inventory

2.1 EC2 — g4dn.xlarge (GPU Compute) AWAITING GPU QUOTA

Instance Specification

Property	Value
Instance Type	g4dn.xlarge — 4 vCPU, 16 GB RAM, NVIDIA T4 (16 GB VRAM)
AMI	ami-0e1c254c6491f13f2 — Amazon Linux 2023, kernel 6.18, NVIDIA drivers ready
Storage	125 GB gp3 (root EBS) — holds Docker images, conda env, training data cache
Key Pair	p053-key — stored at ~/.ssh/p053-key.pem (chmod 400)
Cost	$0.526/hr on-demand — auto-stop coded via ec2_auto_stop.py
Blocker	GPU quota = 0 vCPUs for G/P instances — increase to 4 requested (Service Quotas)

GPU Auto-Selector (gpu_selector.py): Automatically selects GPU tier based on model params + data volume:

Trigger	GPU	Instance	Cost/hr
<50M params AND <100M rows	T4 16GB	g4dn.xlarge	$0.53
50M–500M params OR 100M–1B rows	V100 16GB	p3.2xlarge	$3.06
500M–1.2B params	A10G 24GB	g5.2xlarge	$1.21
>1.2B params OR >1B rows	A100 80GB	p4d.24xlarge	$32.77

Our 317K-parameter model always selects T4. The selector logs "CURRENT GPU SUFFICIENT" in MLflow. If model size grows, it auto-flags "SWITCH REQUIRED" with cost estimate.

2.2 RDS — PostgreSQL 17 STOPPED

Database Specification

Property	Value
Identifier	p053-mlflow-db
Endpoint	p053-mlflow-db.cxmsugggu12o.us-west-2.rds.amazonaws.com:5432
Engine	PostgreSQL 17, default.postgres17 parameter group
Instance Class	db.t3.micro — 2 vCPU, 1 GB RAM
Storage	20 GB gp2, auto-backup window 10:12–10:42 UTC, 7-day retention
VPC	vpc-0434579799a7af5a5, Availability Zone: us-west-2d
Security Group	sg-0f11ba29c1155cba3 (shared with EC2)
Cost (when running)	$0.018/hr = $13/month
Created	2026-04-07T11:00:22 UTC
Master Username	mlflow / Database: mlflow

⚠️ **Auto-Restart Warning:** AWS auto-restarts stopped RDS instances after 7 days. If waiting >7 days for GPU quota, you must re-stop it. Use `bash check_aws_services.sh` to check status any time.

Password note: If your password contains special characters (like `@`), URL-encode them in the connection string: `@` → `%40`. Best practice: use alphanumeric-only password to avoid encoding issues.

2.3 S3 — Artifact Storage ACTIVE

Bucket: p053-mlflow-artifacts

Property	Value
Versioning	Enabled — every model version preserved
Region	us-west-2
Cost	$0.023/GB/month (Standard). Current: <$0.01/month
Artifacts uploaded	models/day1_champion.pt (Day 1 A100 champion, AUC-ROC=0.816)
Access	Private — IAM role access from EC2 only via instance profile

2.4 ECR — Docker Image Registry ACTIVE

Repository: 053-memory-yield-predictor

Property	Value
URI	718036735422.dkr.ecr.us-west-2.amazonaws.com/053-memory-yield-predictor
Image	p053-airflow-gpu — PyTorch 2.2 + CUDA 12.1 + Airflow 2.8
Build trigger	GitHub Actions on git tag push (v1.x.x)
Cost	$0 for first 500 MB/month, then $0.10/GB

2.5 Security Group & Networking ACTIVE

Security Group: p053-sg (sg-0f11ba29c1155cba3)

Port	Protocol	Source	Service
22	TCP	Your IP /32	SSH access to EC2
5001	TCP	Your IP /32	MLflow Tracking UI
8000	TCP	Your IP /32	FastAPI inference server
8080	TCP	Your IP /32	Apache Airflow Web UI
3000	TCP	Your IP /32	Grafana dashboards
9090	TCP	Your IP /32	Prometheus metrics

Dynamic IP: Your home IP changes between sessions. Check each time with `curl -s checkip.amazonaws.com`. If changed, update security group rules (revoke old IP, authorize new IP for all 6 ports). Last known IP: 119.234.92.99.

Network path: Your MacBook → Security Group → EC2 Public IP → Docker internal network → services (MLflow, Airflow, FastAPI). RDS is on the same VPC security group — EC2 reaches it on port 5432 internally.

RDS-Managed Elastic IP: You will see an EIP (34.214.51.57) in your account tagged `ServiceManaged: rds`. This is automatically created and managed by RDS for its VPC connectivity. Do NOT release it manually — AWS will release it when RDS is deleted. It does not incur separate charges.

2.6 CloudWatch — Billing Alarm CODE READY, ACTIVATES ON EC2 LAUNCH

Coded in `src/ec2_auto_stop.py` — `setup_billing_alarm(threshold_usd=200.0)`. This is called during EC2 bootstrap and creates a CloudWatch alarm in `us-east-1` (billing metrics only exist in us-east-1) that SNS-notifies when monthly estimated charges exceed $200 USD.

At $0.526/hr (g4dn.xlarge) + $0.018/hr (RDS) = $0.544/hr combined, reaching $200 would require 368 hours of continuous runtime. Given our simulation runs <8 hours, the real risk is forgetting to stop instances — which the $200 alarm and ec2_auto_stop.py both guard against.

3 Setup Walkthrough

3.1 Prerequisites (All Completed)

> **Status:** Steps 0–8 complete. Blocked at Step 9 (EC2 launch) pending GPU quota approval.

Step	Action	Status
0	Verify AWS CLI: `aws sts get-caller-identity` → Account 718036735422	✅ Done
1	Create IAM user p053-cicd-user with EC2/S3/RDS/ECR/IAM policies	✅ Done
2	Create S3 bucket p053-mlflow-artifacts + enable versioning	✅ Done
3	Create ECR repository 053-memory-yield-predictor	✅ Done
4	Create EC2 key pair p053-key → ~/.ssh/p053-key.pem (chmod 400)	✅ Done
5	Create security group p053-sg, add 6 port rules from your IP	✅ Done
6	Create RDS PostgreSQL 17 (db.t3.micro) — p053-mlflow-db	✅ Done
7	Stop RDS while waiting for GPU quota (saves $0.018/hr)	✅ Done
8	Request GPU quota increase (Service Quotas → G and VT → 4 vCPUs)	⏳ Pending

3.2 Phase 2 — EC2 + RDS Provisioning (When Quota Approved)

1. **Check IP:** `curl -s checkip.amazonaws.com` — if changed from 119.234.92.99, update security group (see Section 8 commands).
2. **Start RDS:** `aws rds start-db-instance --db-instance-identifier p053-mlflow-db --region us-west-2` — wait ~5 min for status "available".
3. **Launch EC2:** g4dn.xlarge with ec2-user-data.sh bootstrap script. Public IP assigned automatically (no need to allocate EIP manually).
4. **Wait ~40 min:** User-data runs: installs Docker + NVIDIA drivers + nvidia-container-toolkit, clones repo, builds GPU Airflow image, starts docker compose. Zero manual SSH needed.
5. **Verify Airflow UI:** `curl http://<EC2_IP>:8080/health` → should return healthy.
6. **Verify MLflow:** `curl http://<EC2_IP>:5001/api/2.0/mlflow/experiments/list` → should list experiments with RDS backend.
7. **Register Day 1 Champion:** Run MLflow registration script against RDS (day1_champion.pt already on S3).

8. **Trigger simulation:** Enable dag_simulation_master DAG → triggers 40-day pipeline automatically.

3.3 Phase 3 — 40-Day Simulation (Autonomous on EC2)

> **MacBook can be OFF:** Once simulation is triggered, EC2 runs autonomously. Airflow DAGs execute on schedule. Training happens on T4 GPU inside EC2. MLflow writes to RDS. S3 receives artifacts. After Phase 3 completes, ec2_auto_stop.py stops EC2 + RDS + NAT gateway automatically.

Day Range	What Happens	EC2 Activity
Days 1–19	Stable production — no drift. Data accumulation.	Airflow DAG runs drift check, logs "no action needed"
Day 20	Moderate drift begins (PSI=0.12). Below retrain threshold (0.25).	Drift detected, logged to MLflow, no retrain triggered
Day 31	Severe drift (PSI=0.28). All 3 criteria met → retrain triggered.	train.py runs on T4 GPU (~4.4 min/epoch × 30 epochs)
Day 31+	Challenger model compared to champion (canary). If AUC-PR improved → promote.	MLflow Model Registry: champion replaced
Day 39	Recovery — second retrain for fine-tuning.	Another T4 GPU training run
Day 40 (end)	Simulation complete. Auto-stop triggered.	EC2 + RDS + NAT → stopped/deleted

4 GPU Quota — Issue & Resolution

> **Current Status:** Quota = 0 vCPUs for "Running On-Demand G and VT instances". Increase to 4 vCPUs requested via AWS Console (root account). Waiting for approval (15 min to 24 hrs). Once approved, EC2 launch proceeds with same command — no changes needed.

Why this happens: AWS defaults all new accounts to 0 GPU vCPUs as a fraud prevention measure. GPU instances cost $0.53–$32/hr, and without limits, a compromised account could accumulate thousands of dollars in minutes.

In enterprise: GPU quotas are pre-negotiated with AWS account teams during the annual Enterprise Discount Program (EDP) commitment. A company committing $1M/yr to AWS typically gets 100+ GPU vCPUs pre-approved. This is why ML teams at AMD/NVIDIA/Qualcomm don't hit this wall — their accounts are pre-configured.

Verification commands:

```
# Check quota status (may return AccessDenied if using IAM user — use root/console instead)
aws service-quotas get-service-quota   --service-code ec2   --quota-code L-DB2E81BA   --region us-west-2

# Test with dry-run (works even without GPU quota)
aws ec2 run-instances --image-id ami-0e1c254c6491f13f2   --instance-type t3.micro --key-name p053-key   --security-group-ids sg-0f11ba29c1155cba3 --dry-run
```

The `--dry-run` with t3.micro returned "DryRunOperation" (would succeed) — confirming account is verified, only the GPU quota is missing.

5 Cost Tracking & Control

Service	Rate	Estimate (full sim)	Control Mechanism
EC2 g4dn.xlarge	$0.526/hr	~8 hrs active = $4.21	ec2_auto_stop.py + CloudWatch alarm
RDS db.t3.micro	$0.018/hr	~8 hrs active = $0.14 + 20GB storage = $0.05/mo	Stopped when idle, auto-stop after Phase 3
S3 (artifacts)	$0.023/GB/mo	<$0.01	Versioned but small (<500 MB)
ECR (images)	$0.10/GB/mo	<$0.05	First 500MB free
NAT Gateway	$0.045/hr	Not yet created	Deleted by ec2_auto_stop.py after Phase 3
Data Transfer	$0.09/GB out	<$0.10	Minimal — only monitoring + S3 pulls
Colab Pro (Day 1 A100)	$10/mo	$10 (one-time day 1 only)	Done — not recurring
TOTAL		**~$72 USD**	CloudWatch alarm at $200

Check current spend:

```
# Check services status anytime
bash check_aws_services.sh

# Check S3 bucket size
aws s3api list-objects-v2 --bucket p053-mlflow-artifacts   --query
'sum(Contents[].Size)' --output text
```

6 Security Practices

- **No secrets in code:** All credentials in `.env.aws` (gitignored). GitHub Actions uses GitHub Secrets for AWS keys.
- **IP-locked security group:** All 6 ports restricted to your IP /32. Reduces attack surface vs 0.0.0.0/0.
- **IAM least privilege:** p053-cicd-user has only EC2/S3/RDS/ECR/IAM policies — not root. CI/CD uses this user for ECR push.

- **No hardcoded credentials in Docker:** EC2 instance profile (IAM role) used for S3/ECR access from EC2. No AWS keys on the instance.
- **RDS not public:** RDS is in the same VPC as EC2. EC2 reaches it at private IP. RDS does not need a public endpoint — all traffic stays inside AWS.
- **Database password:** Strong password, URL-encoded if it contains @ (replace @ with %40 in connection string). Stored in .env.aws (gitignored), not in source code.
- **.gitignore protects:** `deploy/aws/.env*`, `*.pem`, `__pycache__`, `mlruns/`, `mlartifacts/` all excluded from git.

7 Operational Procedures

7.1 Stop Everything (Save Cost)

```
# Stop RDS (saves $0.018/hr while waiting for quota):
aws rds stop-db-instance --db-instance-identifier p053-mlflow-db --region us-west-2

# Stop EC2 (run after SSH in and stopping docker compose):
aws ec2 stop-instances --instance-ids <INSTANCE_ID> --region us-west-2

# Verify everything is stopped:
bash check_aws_services.sh
```

7.2 Resume Session (When GPU Quota Approved)

1. Check your IP: `curl -s checkip.amazonaws.com`
   If changed from 119.234.92.99: update security group (revoke old IP for all 6 ports, authorize new IP)
2. Start RDS: run start command → wait until status = "available" (~5 min)
3. Launch EC2 g4dn.xlarge with user-data bootstrap script
4. Wait ~40 min for bootstrap to complete
5. Verify services: MLflow UI, Airflow UI, FastAPI health check
6. Register Day 1 champion model in MLflow Model Registry
7. Enable simulation master DAG → walk away (MacBook can be off)
8. Check status from AWS Console mobile app or check_aws_services.sh

7.3 MacBook Requirement

> **Short answer: NO — your MacBook does NOT need to stay on once the simulation is triggered.**

When you launch the Airflow DAG on EC2:

- All compute runs on EC2 g4dn.xlarge in AWS Oregon (us-west-2)
- Airflow schedules and executes DAGs autonomously on the EC2 instance
- GPU training runs inside EC2 (T4) — the subprocess call to train.py runs inside Docker, on EC2
- MLflow writes to RDS (also on AWS) — no MacBook involvement
- S3 artifact saves happen from EC2 to S3 — all within AWS network
- ec2_auto_stop.py runs inside the Airflow DAG on EC2 — stops everything when done

You only need your MacBook for: triggering the DAG (one browser tab), and checking status (AWS Console mobile app or check_aws_services.sh). You can sleep, commute, or work on other

things while the 2–3 hour simulation runs.

Monitoring from phone: Install "AWS Console" app (free, App Store). Shows EC2 status, RDS status, CloudWatch alarms, and S3. You'll get notified if the $200 billing alarm triggers.

7.4 RDS-Managed Elastic IPs

Your `check_aws_services.sh` will show an Elastic IP (34.214.51.57, eipalloc-0ae0823a57419a290) with `ServiceManaged: rds`. This is **completely normal and safe** — AWS RDS automatically allocates and manages this EIP for VPC connectivity. It is:

- Created automatically when RDS was created — you did not allocate it
- Managed by AWS — you cannot and should not release it manually
- Released automatically when RDS is deleted
- Not billed separately — included in RDS pricing

The check script now labels these as `[AUTO-MANAGED by RDS]` and suppresses the billing warning for them.

8 Quick Reference Commands

Status & Monitoring

```
# Everything at once:
bash check_aws_services.sh

# RDS status only:
aws rds describe-db-instances   --db-instance-identifier p053-mlflow-db   --query
'DBInstances[0].DBInstanceStatus' --output text --region us-west-2

# EC2 status (after launch):
aws ec2 describe-instances   --filters "Name=tag:Name,Values=p053-mlflow"   --query
'Reservations[0].Instances[0].State.Name' --output text --region us-west-2
```

IP Update (When Your IP Changes)

```
# Get current IP:
MY_IP=$(curl -s checkip.amazonaws.com)
SG=sg-0f11ba29c1155cba3

# Revoke old IP for all 6 ports:
OLD_IP="119.234.92.99"
for PORT in 22 3000 5001 8000 8080 9090; do
  aws ec2 revoke-security-group-ingress --group-id $SG     --protocol tcp --port $PORT -
-cidr ${OLD_IP}/32 --region us-west-2
done

# Authorize new IP:
for PORT in 22 3000 5001 8000 8080 9090; do
  aws ec2 authorize-security-group-ingress --group-id $SG     --protocol tcp --port
$PORT --cidr ${MY_IP}/32 --region us-west-2
done
echo "Updated all 6 ports to $MY_IP"
```

RDS Start / Stop

```
# Stop RDS (saves money while idle):
aws rds stop-db-instance --db-instance-identifier p053-mlflow-db --region us-west-2

# Start RDS (when resuming for EC2 launch):
aws rds start-db-instance --db-instance-identifier p053-mlflow-db --region us-west-2

# Wait for RDS to be available (~5 min):
aws rds wait db-instance-available   --db-instance-identifier p053-mlflow-db --region
```

```
us-west-2
echo "RDS is ready!"
```

MLflow Tracking URI

```
# Connection string format (replace YOUR_PASSWORD — URL-encode @ as %40):
postgresql://mlflow:YOUR_PASSWORD@p053-mlflow-db.cxmsugggu12o.us-west-
2.rds.amazonaws.com:5432/mlflow

# Set as environment variable:
export MLFLOW_TRACKING_URI="postgresql://mlflow:YOUR_PASSWORD@p053-mlflow-
db.cxmsugggu12o.us-west-2.rds.amazonaws.com:5432/mlflow"
```

S3 Operations

```
# List all artifacts:
aws s3 ls s3://p053-mlflow-artifacts/ --recursive --human-readable

# Download day1 champion model:
aws s3 cp s3://p053-mlflow-artifacts/models/day1_champion.pt ./

# Check bucket size:
aws s3api list-objects-v2 --bucket p053-mlflow-artifacts   --query
'sum(Contents[].Size)' --output text
```

ECR Login (Before Docker Push)

```
# Login to ECR:
aws ecr get-login-password --region us-west-2 |   docker login --username AWS --
password-stdin   718036735422.dkr.ecr.us-west-2.amazonaws.com
```